// README · MODULE DOCUMENTATION

ARERA Domestic *Consumption* Aggregator

Processes raw monthly electricity withdrawal data for domestic clients sourced from the Italian Regulatory Authority for Energy, Networks and Environment (ARERA). Consolidates 22 provincial Excel files into a single relational dataset for PostgreSQL ingestion.

`Python 3.11` `pandas` `openpyxl`

Regional scope: This module is **shared across all three regions**. It processes Puglia (6 provinces), Calabria (5 provinces), and Sicilia (9 provinces) simultaneously — 22 provinces in total.

FILE	LANGUAGE	OUTPUT
ARERA_Domestic_Consumption_AGGREGATOR/	Python	Prelievo_Medio_Consolidato.xlsx

PROVINCES COVERED

22

// STEP 01

Configuration & ISTAT Code Mapping

All static configuration is declared at the top of the script. The central data structure is a dictionary mapping each province name to its official ISTAT COD_PROV numeric code. These codes serve as foreign keys linking this dataset to spatial tables in PostgreSQL.

1.1 **Province Code Dictionary (22 provinces)**

Three regional groups are defined in one dictionary:

```
PROVINCE_CODES = {
    # Puglia (6)
    'Foggia': 71, 'Bari': 72, 'Taranto': 73,
    'Brindisi': 74, 'Lecce': 75, 'Barletta-Andria-Trani': 110,
    # Calabria (5)
    'Cosenza': 78, 'Catanzaro': 79, 'Reggio di Calabria': 80,
    'Crotone': 101, 'Vibo Valentia': 102,
    # Sicilia (9)
    'Trapani': 81, 'Palermo': 82, ...
}
```

1.2 **Output Schema Definition**

The 12 monthly column names and the file prefix used by ARERA's raw export format are defined as constants, ensuring the extraction logic is decoupled from the column naming convention.

```
MONTHS_COLS = ['jan_kwh', 'feb_kwh', ..., 'dec_kwh']
FILE_PREFIX = "Bar chart colour gradient vertical - Annomese 2 "
```

// STEP 02

Data Extraction & Processing

2.1 **File Discovery via Glob Pattern**

The script searches the working directory for all files matching `FILE_PREFIX + "*.xlsx"`. Each matching file corresponds to one province. The province name is extracted by stripping the fixed prefix and the `.xlsx` extension from the filename.

2.2 **ISTAT Code Assignment**

The extracted province name is looked up in `PROVINCE_CODES`. If not found (e.g. a naming mismatch), the code defaults to **0** as a sentinel value for manual review.

2.3 **Monthly kWh Extraction**

The raw ARERA Excel file is read with `pd.read_excel()`. The script assumes **chronological order**: rows 0–11 correspond to January–December. The monthly kWh value is taken from **column index 1** (second column) of each row. Non-numeric cells are caught and replaced with 0.0.

```
for i in range(min(12, len(df))):
    kwh = float(df.iloc[i, 1])  # row i, col 1
    row_data[MONTHS_COLS[i]] = kwh
    yearly_total += kwh
```

2.4 **Annual Integral**

The **yearly_kwh** field is the simple sum of all 12 monthly values. This represents the average annual per-capita domestic electricity withdrawal in kWh for that province, which becomes the denominator for residential building-level consumption in the Demand Module.

// STEP 03

Export Consolidated Dataset

3.1 **DataFrame Assembly & Column Ordering**

All province row dictionaries are assembled into a single pandas DataFrame. Columns are reordered to a logical schema: identifiers first, monthly values in calendar order, annual total last.

3.2 **Excel Export**

The consolidated DataFrame is written to `Prelievo_Medio_Consolidato.xlsx`. This file is subsequently loaded into PostgreSQL as the table `prelievo_medio_dei_clienti_domestici`, which the Demand Module joins via `cod_prov` to assign per-capita kWh to buildings.

OUTPUT FILE: PRELIEVO_MEDIO_CONSOLIDATO.XLSX

`code_prov`	ISTAT provincial code (integer)
`den_uts`	Province name (string)
`year`	Reference year (2024)
`jan_kwh … dec_kwh`	Monthly per-capita withdrawal (kWh/month)
`yearly_kwh`	Annual per-capita withdrawal — used by Demand Module

How this feeds the pipeline: The Demand Module joins `prelievo_medio_dei_clienti_domestici.yearly_kwh` to each building's active dwelling count (`active_dwellings_21`) to compute `cons_res_kwh_yr = active_dwellings × yearly_kwh`.

📝
Note: The initial draft of this README documentation was generated with the assistance of Claude AI and subsequently reviewed, verified, and edited by the author for accuracy.